

{ } JavaScript Short Notes

beginner → advanced · interview-ready · personal revision notebook

Contents

01 Basics · 02 Variables · 03 Data Types · 04 Operators · 05 Type Conversion · 06 Strings · 07 Arrays · 08 Objects · 09 Functions · 10 Scope · 11 Hoisting · 12 Closures · 13 this Keyword · 14 Prototypes & OOP · 15 Async JS · 16 Event Loop · 17 DOM · 18 Events · 19 ES6+ · 20 Modules · 21 Error Handling · 22 JSON · 23 Browser Storage · 24 Fetch API · 25 Memory · 26 Performance · 27 Comparisons · 28 Cheat Sheet

read once before every interview ★

01. JavaScript Basics

What is JS?

High-level, interpreted, dynamically-typed scripting language for the web. Runs in browser + Node.js.

Features

Dynamic typing · prototype-based OOP · first-class functions · single-threaded · event-driven · async via event loop.

Uses

Web pages (DOM), servers (Node.js), mobile (React Native), desktop (Electron).

ECMAScript

The specification/standard JS follows. ES6 (2015) introduced let, const, arrow fn, classes, promises.

JS Engine

Program that executes JS. Parses code → converts to bytecode/machine code → runs it.

V8

Google's JS engine (Chrome, Node.js). Compiles JS to machine code using JIT for speed.

How JS executes

Parse → Compile (JIT) → Execute, inside a single call stack, one thing at a time.

Single-threaded

JS has ONE call stack → executes one line at a time. Async tasks handled via event loop, not extra threads.

Interpreted / JIT

Historically interpreted line-by-line; modern engines use JIT (Just-In-Time) compilation → compiles hot code to machine code at runtime for speed.

Caution: JS is single-threaded but NOT blocking — async ops (`setTimeout`, `fetch`) are handled outside the main thread by the browser/Node APIs.

02. Variables

var

Function-scoped, hoisted & initialized as undefined, can be redeclared.

```
var x = 10;
```

let

Block-scoped, hoisted but NOT initialized (TDZ), cannot redeclare in same scope.

```
let x = 10;
```

const

Block-scoped, must be initialized at declaration, cannot be reassigned (object contents CAN still change).

```
const x = 10;
```

Declaration vs Initialization

Declaration = creating the variable (let x;). Initialization = giving it a first value (x = 5;).

Scope

Where a variable is accessible from.

Global scope

Declared outside any function/block — accessible everywhere.

Function scope

var is scoped to the nearest enclosing function.

Block scope

let/const scoped to nearest { } block (if, for, while, etc).

Hoisting

JS moves declarations to the top of their scope before execution.

TDZ (Temporal Dead Zone)

Time between entering scope and the actual let/const declaration line — accessing the variable here throws a ReferenceError.

var vs let vs const

	var	let	const
Scope	function	block	block
Hoisted	yes (=undefined)	yes (TDZ)	yes (TDZ)
Redeclare	yes	no	no
Reassign	yes	yes	no

Tip: Default to const. Use let only if you need to reassign. Avoid var in modern code.

03. Data Types

Primitive types

String · Number · Boolean · Undefined · Null · BigInt · Symbol — stored by VALUE, immutable.

```
let s="hi"; let n=42; let b=true; let u; let x=null; let big=10n; let sym=Symbol();
```

Non-primitive (reference) types

Object · Array · Function — stored by REFERENCE, mutable.

```
let obj={}; let arr=[]; function f(){} 
```

typeof

Returns the type of a value as a string.

Expression	Result
<code>typeof 'hi'</code>	'string'
<code>typeof 42</code>	'number'
<code>typeof null</code>	'object' (known JS bug!)

Primitive vs Reference

Primitives copy by VALUE (independent copies). Objects copy by REFERENCE (both variables point to same memory).

Mutable vs Immutable

Primitives are immutable (can't change 'hi' itself, only reassign). Objects/arrays ARE mutable (can change contents).

Caution: `typeof null` returns "object" — a long-standing JS bug, not a feature.

04. Operators

- `+ - * / %` — arithmetic
- `= += -= *=` — assignment
- `> < >= <=` — comparison
- `&& || !` — logical AND / OR / NOT
- `cond ? a : b` — ternary (shorthand if/else)
- `typeof, -x, !x` — unary
- `x++ / x--` — increment / decrement
- `a ?? b` — nullish coalescing — b only if a is null/undefined
- `a?.b` — optional chaining — safe access, no error if a is null
- `...arr` — spread — expands array/object into elements
- `(...args)` — rest — collects remaining args into array

== VS ===

`==` compares value only (does type coercion). `===` compares value AND type (strict, preferred).

Expression	Result
<code>0 == '0'</code>	true (coerced)
<code>0 === '0'</code>	false (different type)
<code>null == undefined</code>	true
<code>null === undefined</code>	false

Always use `===` unless you specifically need coercion.

05. Type Conversion & Coercion

Explicit conversion

You manually convert the type.

Expression	Result
<code>String(123)</code>	'123'
<code>Number('123')</code>	123
<code>Boolean(1)</code>	true

Implicit conversion (coercion)

JS automatically converts type during an operation.

Expression	Result
<code>'5' + 1</code>	'51' (number to string)
<code>'5' - 1</code>	4 (string to number)
<code>'5' * '2'</code>	10

parseInt() / parseFloat()

Parse a string and extract a number from the start of it.

Expression	Result
<code>parseInt('42px')</code>	42
<code>parseFloat('3.14m')</code>	3.14

Truthy

Values that act as true in a boolean context: any non-empty string, non-zero number, objects, arrays.

Falsy

false, 0, "", null, undefined, NaN — everything else is truthy.

Expression	Result
<code>if ('') {}</code>	never runs
<code>if ('0') {}</code>	RUNS! '0' is a non-empty string = truthy

Caution: '0', [], '{}' are all truthy — only the 6 falsy values above are false.

06. Strings

Creating strings

Single, double quotes, or template literals.

```
let a='hi'; let b="hi"; let c=`hi ${name}`;
```

Template literals

Backticks allow variable interpolation and multi-line strings.

```
let msg = `Hello, ${user}! Total: ${1+2}`;
```

Common string methods

Method	Description
<code>.length</code>	string length — <code>'hi'.length // 2</code>
<code>.charAt(i)</code>	char at index — <code>'hi'.charAt(0) // 'h'</code>
<code>.includes(s)</code>	true/false if substring exists
<code>.indexOf(s)</code>	first index of substring, -1 if not found
<code>.slice(a,b)</code>	extract part, supports negative index
<code>.substring(a,b)</code>	extract part, no negative index
<code>.replace(a,b)</code>	replace first match
<code>.replaceAll(a,b)</code>	replace all matches
<code>.split(sep)</code>	string → array
<code>.trim()</code>	remove whitespace from both ends
<code>.toUpperCase()</code>	→ ALL CAPS
<code>.toLowerCase()</code>	→ all lowercase

Tip: Strings are immutable — every method RETURNS a new string, doesn't change the original.

07. Arrays

Creating arrays

```
let arr = [1,2,3]; let arr2 = new Array(1,2,3);
```

Index / length

Zero-based index. `.length` = number of elements.

Expression	Meaning
<code>arr[0]</code>	first element
<code>arr.length</code>	number of elements

Array.isArray()

Checks if a value is an array.

```
Array.isArray([1,2]) // true
```

Array methods

Method	Description
<code>.push(x)</code>	add to end (mutates)
<code>.pop()</code>	remove from end (mutates)
<code>.shift()</code>	remove from start (mutates)
<code>.unshift(x)</code>	add to start (mutates)
<code>.slice(a,b)</code>	copy portion (no mutate)
<code>.splice(i,n)</code>	remove/insert (mutates)
<code>.concat(arr2)</code>	merge arrays (no mutate)
<code>.join(sep)</code>	array → string
<code>.includes(x)</code>	true/false if value exists
<code>.indexOf(x)</code>	first index, -1 if absent
<code>.find(fn)</code>	first element matching fn
<code>.findIndex(fn)</code>	index of first match
<code>.filter(fn)</code>	new array of matches
<code>.map(fn)</code>	new array, transformed
<code>.reduce(fn, init)</code>	reduce to single value
<code>.forEach(fn)</code>	loop, no return value
<code>.some(fn)</code>	true if ANY match
<code>.every(fn)</code>	true if ALL match
<code>.sort(fn)</code>	sort (mutates!)
<code>.reverse()</code>	reverse order (mutates)
<code>.flat(depth)</code>	flatten nested arrays

★ map vs filter vs reduce vs forEach

method	returns	use for
map	new array (same length)	transform each element
filter	new array (subset)	keep elements matching condition
reduce	single value	accumulate into total/object/etc
forEach	undefined	just looping, side-effects only

```
[1,2,3].map(x => x*2) // [2,4,6]
[1,2,3].filter(x => x>1) // [2,3]
[1,2,3].reduce((a,b)=>a+b,0) // 6
[1,2,3].forEach(x=>console.log(x)) // logs, returns undefined
```

Caution: *sort() and reverse() and splice() MUTATE the original array — the rest don't.*

08. Objects

Object creation

```
let obj = { name: 'Sam', age: 25 };
```

Dot vs Bracket notation

Bracket allows dynamic/variable keys.

```
obj.name // 'Sam'  
obj['name'] // 'Sam'  
let k='name'; obj[k]
```

Nested objects

```
let user = { address: { city: 'Pune' } };  
user.address.city
```

Object methods

Method	Description
<code>Object.keys(o)</code>	array of keys
<code>Object.values(o)</code>	array of values
<code>Object.entries(o)</code>	array of [key,value] pairs
<code>Object.hasOwn(o,k)</code>	true if key exists on object
<code>Object.assign(a,b)</code>	merge/copy objects
<code>Object.freeze(o)</code>	make immutable (no add/edit/delete)
<code>Object.seal(o)</code>	no add/delete, but CAN edit existing

Destructuring

Extract properties into variables.

```
const { name, age } = obj;  
const [a,b] = [1,2];
```

Spread (objects)

Copy/merge object properties.

```
let copy = { ...obj, age: 26 };
```

Computed properties

Use a variable as a key name.

```
let key='role'; let o = { [key]: 'admin' };
```

Caution: `Object.freeze()` is shallow — nested objects inside can still be mutated.

09. Functions

Function declaration

Hoisted fully — can call before definition.

```
function greet(name){ return 'Hi '+name; }
```

Function expression

Not hoisted like declarations — assigned to a variable.

```
const greet = function(name){ return 'Hi '+name; };
```

Arrow function

Shorter syntax, no own 'this'/arguments'.

```
const greet = (name) => `Hi ${name}`;
```

Parameters vs Arguments

Parameters = named vars in function definition. Arguments = actual values passed in the call.

Default parameters

```
function f(x=10){ return x; }
```

Rest parameters

Collect extra args into an array.

```
function sum(...nums){ return nums.reduce((a,b)=>a+b); }
```

Callback function

A function passed as an argument to run later.

```
setTimeout(() => console.log('done'), 1000);
```

Higher-order function

A function that takes and/or returns another function.

```
arr.map(x => x*2) // map is HOF
```

First-class function

Functions treated as values — can be assigned, passed, returned.

IIFE

Immediately Invoked Function Expression — runs right away, creates private scope.

```
(function(){ console.log('runs now'); })();
```

Recursive function

A function that calls itself.

```
function fact(n){ return n<=1 ? 1 : n*fact(n-1); }
```

Pure function

Same input always → same output, no side effects (doesn't modify outside state).

10. Scope

Global → Function → Block

Scopes nest like Russian dolls — inner scope can access outer, not vice versa.

Lexical scope

Scope is determined by WHERE code is physically written, not where it's called from.

Scope chain

When a variable isn't found in current scope, JS looks up through parent scopes until global.

Variable lookup

JS searches innermost → outward, stops at first match, throws ReferenceError if none found.

```
let a = 1;
function outer(){
  let b = 2;
  function inner(){
    let c = 3;
    console.log(a,b,c); // 1 2 3 – inner sees all outer scopes
  }
  inner();
}
```

11. Hoisting

var hoisting

Declaration moved to top, initialized as undefined.

```
console.log(x); // undefined
var x = 5;
```

let/const hoisting

Declaration is hoisted but NOT initialized — stuck in TDZ until the line runs.

```
console.log(y); // ReferenceError
let y = 5;
```

function hoisting

Entire function declaration (name + body) is hoisted — callable before defined.

```
greet(); // works!
function greet(){ console.log('hi'); }
```

TDZ

Temporal Dead Zone — the period where let/const exist but can't be accessed yet.

Caution: Function EXPRESSIONS (`const f = function(){}`) are NOT hoisted like declarations — only the variable is hoisted (as undefined for var, TDZ for let/const).

12. Closures ★

Meaning

A closure is a function that remembers the variables from its outer (enclosing) scope, even after the outer function has finished running.

```
function outer() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
const counter = outer();  
counter(); // 1  
counter(); // 2
```

Why closures are useful

Data privacy (private variables), function factories, event handlers, callbacks, currying, memoization.

Real-world example

A counter, a debounce function, or module pattern — all rely on closures to "remember" state.

Inner function can access outer function's variables even after the outer function has finished executing.

13. this Keyword

Context	'this' refers to
Global	window (browser) / undefined in strict mode / module
Regular function	window, or undefined in strict mode (NOT the caller)
Object method	the object the method was called on
Arrow function	no own this — inherits this from the surrounding (lexical) scope
Constructor (new)	the newly created object instance
Class	the class instance, like a constructor

```
const obj = {  
  name: 'Sam',  
  greet(){ console.log(this.name); }, // 'Sam'  
  greetArrow: () => console.log(this.name) // undefined, arrow has no own this  
};
```

call()

Invoke a function with a given 'this' + args listed individually.

```
greet.call(obj, 'hi');
```

apply()

Same as call, but args passed as an array.

```
greet.apply(obj, ['hi']);
```

bind()

Returns a NEW function with 'this' permanently bound (doesn't call immediately).

```
const bound = greet.bind(obj);  
bound('hi');
```

Caution: Arrow functions don't have their own 'this' — never use them as object methods if you need 'this' to refer to the object.

14. Prototypes & OOP

Prototype

An object that other objects inherit properties/methods from.

Prototype chain

When a property isn't found on an object, JS looks up its `__proto__` chain until found or reaches null.

`__proto__` vs `.prototype`

`__proto__` = the actual link an object uses for lookup. `.prototype` = property on a FUNCTION, used as `__proto__` for instances it creates.

Constructor function

A regular function used with 'new' to create objects.

```
function Person(name){ this.name=name; }  
const p = new Person('Sam');
```

new keyword

Creates a new empty object → sets its `__proto__` → runs constructor with 'this' = new object → returns it.

Class

Syntactic sugar over prototypes — cleaner OOP syntax.

```
class Person {  
  constructor(name){ this.name = name; }  
  greet(){ return 'Hi ' + this.name; }  
}
```

extends / super

extends = inherit from a parent class. `super()` = call parent constructor/methods.

```
class Student extends Person {  
  constructor(name, grade){  
    super(name);  
    this.grade = grade;  
  }  
}
```

OOP Pillars

- **Inheritance** — child class reuses parent's properties/methods (extends)
- **Encapsulation** — bundling data + methods, hiding internal details (`#privateField`)
- **Polymorphism** — same method name behaves differently in different classes (override)
- **Abstraction** — expose only what's necessary, hide complex implementation

static — method/property on the class itself, not instances — `ClassName.method()`

getter / setter — `get x(){}` / `set x(v){}` — access like a property but run code

Tip: Every object has a prototype except the base `Object.prototype` (which is null) — that's the top of the chain.

15. Asynchronous JavaScript

Sync vs Async

Sync = each line waits for the previous to finish (blocking). Async = code can start a task and move on, result handled later (non-blocking).

Callback

A function passed to run after an async task completes.

```
fetchData(function(result){ console.log(result); });
```

Callback hell

Deeply nested callbacks — hard to read/maintain (pyramid of doom).

Promise

An object representing a future value of an async operation.

```
const p = new Promise((resolve, reject) => { resolve('done'); });
```

Promise states

pending → fulfilled (resolved) OR rejected. Once settled, state never changes again.

Method	Runs on
<code>.then()</code>	success (fulfilled)
<code>.catch()</code>	failure (rejected)
<code>.finally()</code>	always, success or failure

Promise chaining

Return a value/promise inside `.then()` to chain the next `.then()`.

```
fetchUser()  
.then(u => fetchPosts(u.id))  
.then(posts => console.log(posts))  
.catch(err => console.error(err));
```

Method	Behavior
<code>Promise.all([...])</code>	waits for ALL, rejects if ANY fails
<code>Promise.allSettled([...])</code>	waits for ALL, never rejects, gives status of each
<code>Promise.race([...])</code>	resolves/rejects as soon as the FIRST settles
<code>Promise.any([...])</code>	resolves as soon as FIRST succeeds, ignores rejections

async / await

Syntactic sugar over promises — write async code that looks synchronous.

```
async function getData(){  
  try {  
    const res = await fetch(url);  
    const data = await res.json();  
  } catch(err) { console.error(err); }  
}
```

Caution: *await only works inside an async function. Always wrap await in try/catch to handle errors.*

16. Event Loop ★

Loop repeatedly checks: is the call stack empty? If yes → push the next callback from the queue onto the stack.

Call Stack ■ Web APIs ■ Callback Queue ■ Event Loop

The call stack hands off async work to Web APIs; completed work is queued, and the event loop moves it back onto the stack once it's empty.

Microtask queue

Higher priority queue — Promises (.then), queueMicrotask. Fully drained BEFORE next macrotask.

Macrotask queue

Lower priority — setTimeout, setInterval, I/O. One task runs per event loop cycle.

3 output examples

```
console.log(1);
setTimeout(()=>console.log(2), 0);
console.log(3);
// Output: 1, 3, 2 (setTimeout is a macrotask, runs after sync code)

console.log(1);
Promise.resolve().then(()=>console.log(2));
console.log(3);
// Output: 1, 3, 2 (microtask, but still after sync code)

console.log(1);
setTimeout(()=>console.log(2), 0);
Promise.resolve().then(()=>console.log(3));
console.log(4);
// Output: 1, 4, 3, 2 (sync → microtasks → macrotasks)
```

Order: all synchronous code first → then ALL microtasks (promises) → then one macrotask (setTimeout) → repeat.

17. DOM

DOM

Document Object Model — a tree-like JS representation of the HTML page, so JS can read/change it.

DOM tree

html → head/body → nested elements, each a 'node'.

Method / Property	Description
<code>document.getElementById(id)</code>	select by id
<code>document.querySelector(sel)</code>	select FIRST match (CSS selector)
<code>document.querySelectorAll(sel)</code>	select ALL matches (NodeList)
<code>document.createElement(tag)</code>	create a new element node
<code>el.textContent</code>	get/set text only (safe, no HTML parsing)
<code>el.innerHTML</code>	get/set HTML content (careful — XSS risk)
<code>el.style.prop</code>	set inline CSS
<code>el.classList</code>	<code>add()</code> / <code>remove()</code> / <code>toggle()</code> / <code>contains()</code>
<code>el.getAttribute/setAttribute</code>	read/write HTML attributes
<code>el.append(node)</code>	add child at the end
<code>el.prepend(node)</code>	add child at the start
<code>el.remove()</code>	remove element from DOM

Caution: Prefer `textContent` over `innerHTML` when inserting user-provided text — `innerHTML` can execute injected scripts.

18. Events

Event

An action/occurrence the browser can detect — click, input, load, submit, keydown, etc.

addEventListener

Attach a handler function to an event.

```
btn.addEventListener('click', (e) => { console.log('clicked'); });
```

Event bubbling

Event fires on the target, then bubbles UP through ancestors (default behavior).

Event capturing

Event travels DOWN from ancestors to target first (opt-in via 3rd arg: true).

Event propagation

The overall journey of an event — capture phase → target → bubble phase.

stopPropagation()

Stops the event from bubbling/capturing further.

preventDefault()

Stops the browser's default action (e.g. form submit, link navigation).

Event delegation

Attach ONE listener to a parent instead of many children — use e.target to identify which child triggered it. More efficient.

```
list.addEventListener('click', (e) => {  
  if(e.target.tagName === 'LI') console.log(e.target.textContent);  
});
```

19. ES6+ Features — Quick Revision

- **let / const** — block-scoped declarations
- **Arrow fn** — `(x)=>x*2`, no own `this`
- **Template literals** — ``Hi ${name}``
- **Destructuring** — `const {a,b}=obj; const [x,y]=arr;`
- **Spread ...** — expand array/object: `[...arr], {...obj}`
- **Rest ...** — collect into array: `function(...args)`
- **Default params** — `function f(x=10){}`
- **Classes** — `class X { constructor(){} }`
- **Modules** — import / export between files
- **Promises** — async operation wrapper
- **for...of** — loop over iterable VALUES
- **Optional chaining ?.** — safe property access
- **Nullish coalescing ??** — default only for null/undefined

20. Modules

export / import (named)

Export multiple named values, import with matching names.

```
// math.js
export const add=(a,b) => a+b;

// app.js
import { add } from './math.js';
```

export default

One default export per file, import with any name.

```
// math.js
export default function add(a,b){return a+b;}

// app.js
import add from './math.js';
```

ES Modules (ESM)

Modern standard — import/export, static, tree-shakeable. Used in browsers + modern Node.

CommonJS (CJS)

Node's original module system.

```
// export
module.exports = add;

// import
const add = require('./math.js');
```

require vs import

	require (CJS)	import (ESM)
Loading	synchronous, dynamic	static, hoisted
Where	Node.js (default)	browsers & modern Node (type:module)
Syntax	const x = require('x')	import x from 'x'

21. Error Handling

try / catch / finally

Run risky code in try, handle errors in catch, always run finally.

```
try {  
  riskyCall();  
} catch(err) {  
  console.error(err.message);  
} finally {  
  cleanup();  
}
```

throw

Manually raise an error.

```
throw new Error('Something went wrong');
```

Built-in error types

Error type	Meaning
<code>SyntaxError</code>	invalid code structure
<code>TypeError</code>	wrong type used (e.g. calling a non-function)
<code>ReferenceError</code>	using an undeclared variable
<code>RangeError</code>	value out of allowed range (e.g. array length -1)

Custom Error

Extend the built-in Error class for domain-specific errors.

```
class ValidationError extends Error {  
  constructor(msg){ super(msg); this.name='ValidationError'; }  
}
```

22. JSON

JSON

JavaScript Object Notation — lightweight text format for data exchange (looks like a JS object, but keys must be strings).

JSON.stringify()

Convert JS object/array → JSON string.

```
JSON.stringify({a:1}) // '{"a":1}'
```

JSON.parse()

Convert JSON string → JS object.

```
JSON.parse('{"a":1}') // {a:1}
```

JSON vs Object

	JSON	JS Object
Type	text/string	actual in-memory object
Keys	must be double-quoted strings	can be unquoted identifiers
Can hold	data only, no functions	functions, methods, symbols too

23. Browser Storage

localStorage

Key-value storage, persists even after browser closes. ~5-10MB. Same-origin only.

sessionStorage

Same API as localStorage, but cleared when the TAB closes.

cookies

Small (~4KB) data sent with every HTTP request to the server. Used for auth/sessions.

	localStorage	sessionStorage	cookies
Persists	forever (until cleared)	tab session only	set expiry date
Size	~5-10MB	~5-10MB	~4KB
Sent to server	no	no	yes, every request
Access	JS only	JS only	JS + server

```
localStorage.setItem('token', 'abc');  
localStorage.getItem('token');  
localStorage.removeItem('token');
```

24. Fetch API

API

A set of rules that lets applications communicate (e.g. your frontend talking to a backend server).

HTTP methods

GET (read) · POST (create) · PUT (replace) · PATCH (partial update) · DELETE (remove).

fetch()

Browser API to make HTTP requests, returns a Promise.

```
const res = await fetch('/api/users');  
const data = await res.json();
```

Response object

Has .ok, .status, .json(), .text() — fetch does NOT reject on 404/500, only on network failure.

POST with body

```
await fetch('/api/users', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ name: 'Sam' })  
});
```

Error handling

Must manually check res.ok since fetch only rejects on network errors.

```
if(!res.ok) throw new Error('Request failed');
```

CORS

Cross-Origin Resource Sharing — browser security that blocks requests to a different origin unless the server explicitly allows it via headers.

Caution: fetch() does NOT throw on a 404 or 500 response — always check res.ok manually.

25. Memory

Stack

Stores primitive values & function call frames. Fixed size, fast, LIFO order.

Heap

Stores objects/arrays/functions — dynamic size, slower, accessed via reference from the stack.

Reference

A variable holding an object doesn't hold the object itself — it holds a pointer/address to it in the heap.

Garbage collection

JS engine automatically frees memory no longer reachable/referenced by any variable.

Memory leak

Memory that's no longer needed but still referenced somewhere, so GC can't free it (e.g. forgotten timers, detached DOM nodes, growing caches).

Shallow copy

Copies only the first level — nested objects are still shared by reference.

```
let copy = {...obj}; // or Object.assign({}, obj)
```

Deep copy

Copies ALL levels, fully independent.

```
let deep = structuredClone(obj);  
// or JSON.parse(JSON.stringify(obj)) – loses functions/undefined
```

Caution: *JSON.parse(JSON.stringify(x)) is a common deep-copy trick, but it drops functions, undefined, and Dates become strings.*

26. Performance

Debouncing

Delay running a function until after a pause in events (e.g. wait 300ms after user stops typing before searching).

```
function debounce(fn, delay){
  let t;
  return (...a) => { clearTimeout(t); t=setTimeout(() => fn(...a), delay); };
}
```

Throttling

Run a function at most once every X ms, no matter how often the event fires (e.g. scroll handler).

```
function throttle(fn, limit){
  let wait=false;
  return (...a) => { if(!wait){ fn(...a); wait=true; setTimeout(() => wait=false, limit); } };
}
```

Lazy loading

Load resources (images, components, routes) only when actually needed, not upfront.

Efficient DOM manipulation

Batch DOM updates, use DocumentFragment, avoid layout thrashing (read then write, don't interleave).

Avoid unnecessary loops

Avoid nested loops over large data ($O(n^2)$) — use maps/sets/objects for $O(1)$ lookups instead.

Caching (memoization)

Store results of expensive function calls, return cached result for repeated same inputs.

```
function memoize(fn){
  const cache = {};
  return (x) => cache[x] ?? (cache[x]=fn(x));
}
```

Tip: Debounce = wait for silence (search box). Throttle = steady rate limit (scroll/resize).

27. Important Comparisons

var vs let vs const

var=function-scoped, redeclarable. let=block-scoped, reassignable. const=block-scoped, not reassignable.

== vs ===

== coerces type before comparing. === compares value AND type, no coercion.

null vs undefined

undefined = variable declared but not assigned. null = explicitly assigned 'no value' by the developer.

map vs filter

map transforms every element (same length out). filter keeps only matching elements (shorter/equal length out).

map vs forEach

map returns a NEW array. forEach returns undefined, used only for side effects.

slice vs splice

slice copies a portion, doesn't mutate. splice removes/inserts in place, DOES mutate the original.

find vs filter

find returns the FIRST matching element (or undefined). filter returns an array of ALL matches.

for...in vs for...of

for...in iterates over KEYS/indexes (objects, arrays). for...of iterates over VALUES (arrays, strings, maps, sets).

regular function vs arrow function

Regular has own this/arguments, hoisted, can be a constructor. Arrow has no own this, not hoisted, can't be 'new'ed.

callback vs promise

Callback = function passed to run later, can lead to nesting/callback hell. Promise = object representing a future value, chainable, better error handling.

promise vs async/await

Promise = .then()/catch() chains. async/await = same thing, written to look synchronous, easier to read.

localStorage vs sessionStorage

localStorage persists forever. sessionStorage clears when the tab closes. Same API otherwise.

shallow copy vs deep copy

Shallow copies only top-level (nested objects shared by reference). Deep copies every level (fully independent).

call vs apply vs bind

call(this, a,b) invokes immediately with listed args. apply(this,[a,b]) invokes immediately with an array of args. bind(this) returns a new function for later use.

prototype vs __proto__

.prototype exists on CONSTRUCTOR FUNCTIONS, used to build __proto__ for new instances. __proto__ exists on every OBJECT, the actual link used for lookup.

28. JavaScript Quick Cheat Sheet

Important Syntax

Feature	Syntax
let/const	<code>let x=1; const y=2;</code>
arrow fn	<code>(a,b) => a+b</code>
template literal	<code>`Hi \${name}`</code>
destructure	<code>const {a,b}=obj; const [x,y]=arr;</code>
spread	<code>[...arr], {...obj}</code>
rest	<code>function f(...args){}</code>
ternary	<code>cond ? a : b</code>
optional chain	<code>obj?.prop?.deep</code>
nullish coalesce	<code>val ?? 'default'</code>
class	<code>class X { constructor(){ } method(){ } }</code>
for...of	<code>for(const v of arr){}</code>
for...in	<code>for(const k in obj){}</code>

Common Operators

`=== / !==` · `??` nullish · `?.` optional chain · `...spread/rest` · `&& / || / !` · `typeof`

Array Methods

`push` · `pop` · `shift` · `unshift` · `slice` · `splice` · `map` · `filter` · `reduce` · `forEach` · `find` · `findIndex` · `includes` · `indexOf` · `some` · `every` · `sort` · `reverse` · `flat` · `join` · `concat`

String Methods

`length` · `charAt` · `includes` · `indexOf` · `slice` · `substring` · `replace` · `replaceAll` · `split` · `trim` · `toUpperCase` · `toLowerCase`

Object Methods

`Object.keys()` · `Object.values()` · `Object.entries()` · `Object.assign()` · `Object.freeze()` · `Object.seal()` · `hasOwn()`

Promise / Async Syntax

`new Promise((res,rej)=>{})` · `.then()` / `.catch()` / `.finally()` · `Promise.all([...])` · `Promise.allSettled([...])` · `Promise.race([...])` · `Promise.any([...])` · `async/await` + `try/catch`

DOM Methods

`getElementById()` · `querySelector()` · `querySelectorAll()` · `createElement()` · `textContent` · `innerHTML` · `classList.add/remove/toggle` · `append()/prepend()` · `remove()`

Event Methods

`addEventListener(evt,fn)` · `e.stopPropagation()` · `e.preventDefault()` · `e.target` · event delegation

last check before interview ★

`closures` · `this` keyword · event loop · promises vs `async/await` · prototype chain · `==` vs `===` · `debounce` vs `throttle`
— if you can explain all of these without notes, you're ready. —